Python Interview Questions & Answers

55 questions with short answers and code examples.
The first 5 are from the Reel; the next 50 are the bonus set.

From the Reel (5 questions)
Core Language (25 questions)
Memory, Performance and Concurrency (25 questions)

From the Reel

1. What's the difference between a list and a tuple?

A list is mutable (you can add, remove, or change items); a tuple is immutable. Tuples are slightly faster and smaller in memory, and can be used as dict keys or set members if all their items are hashable. Use lists for collections that change, tuples for fixed records.

```
a = [1, 2]; a.append(3)      # fine
t = (1, 2); t[0] = 9         # TypeError
```

2. What is the GIL, and why does it matter?

The Global Interpreter Lock is a mutex in CPython that lets only one thread execute Python bytecode at a time. It keeps memory management (reference counting) thread-safe. It matters because CPU-bound code gains nothing from threads; use multiprocessing for CPU work, and threads or asyncio for I/O-bound work, where the GIL is released while waiting.

3. What are decorators, and how do you write one?

A decorator is a function that takes a function and returns a new function, adding behaviour without changing the original code. The @name syntax is shorthand for func = name(func). Use functools.wraps to preserve the wrapped function's metadata.

```
import functools, time

def timer(fn):
    @functools.wraps(fn)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = fn(*args, **kwargs)
        print(f'{fn.__name__}: {time.perf_counter()-start:.4f}s')
        return result
    return wrapper

@timer
def work(): ...
```

4. What's the difference between a shallow copy and a deep copy?

A shallow copy creates a new outer object but reuses references to the nested objects, so changes to nested items show up in both copies. A deep copy recursively copies everything, so the copies are fully independent.

```
import copy
a = [[1, 2], [3]]
b = copy.copy(a)
c = copy.deepcopy(a)
a[0].append(99)
print(b[0])   # [1, 2, 99]  shared
print(c[0])   # [1, 2]      independent
```

5. **What are generators, and why use yield?**

   A generator is a function that uses yield to produce values lazily, one at a time, pausing and resuming its state between calls. It uses constant memory regardless of how many items it produces, which makes it ideal for large or infinite streams.

```
def read_lines(path):
    with open(path) as f:
        for line in f:
            yield line.strip()

for line in read_lines('big.log'):
    ...
```

Core Language

6. What is the difference between mutable and immutable types?

Mutable objects can be changed in place after creation (list, dict, set, bytearray). Immutable objects cannot (int, float, str, tuple, frozenset, bytes). Immutability makes objects hashable and safe to share, but 'changing' one actually creates a new object.

7. What is the difference between == and is?

== compares values (calls __eq__). is compares identity, meaning whether both names point to the exact same object in memory. Use is only for singletons such as None, True, and False.

```
a = [1, 2]; b = [1, 2]
a == b   # True
a is b   # False
x is None   # correct way to check None
```

8. What are *args and **kwargs?

*args collects extra positional arguments into a tuple; **kwargs collects extra keyword arguments into a dict. They let a function accept a flexible number of arguments, and the same symbols can be used to unpack when calling.

```
def f(*args, **kwargs):
    print(args, kwargs)

f(1, 2, a=3)        # (1, 2) {'a': 3}
f(*[1, 2], **{'a': 3})
```

9. List comprehension vs generator expression: what's the difference?

A list comprehension [x for x in data] builds the whole list in memory immediately. A generator expression (x for x in data) produces items lazily. Use a generator expression when you only iterate once or the data is large.

```
sum([x*x for x in range(10**7)])   # builds a big list
sum(x*x for x in range(10**7))     # constant memory
```

10. How does a Python dictionary work internally?

A dict is a hash table. Keys are hashed to find a slot, giving average O(1) lookup, insert, and delete. Keys must be hashable. Since Python 3.7, dicts preserve insertion order. Hash collisions are resolved with open addressing.

11. Why is checking membership in a set faster than in a list?

A set uses hashing, so x in my_set is O(1) on average. A list must scan elements one by one, which is O(n). If you do many membership checks, convert the list to a set first.

12. What is the mutable default argument pitfall?

Default argument values are evaluated once, when the function is defined, not on each call. A mutable default such as [] is therefore shared across calls. Use None as the default and create the object inside the function.

```
def bad(x, acc=[]):
    acc.append(x); return acc
bad(1); bad(2)   # [1, 2] - surprise!

def good(x, acc=None):
    acc = [] if acc is None else acc
    acc.append(x); return acc
```

13. What is the LEGB rule?

It is the order Python uses to resolve names: Local, Enclosing (outer functions), Global (module), then Built-in. The first scope that contains the name wins.

14. What do the global and nonlocal keywords do?

global lets a function rebind a module-level variable. nonlocal lets a nested function rebind a variable in the nearest enclosing function scope. Without them, assignment creates a new local variable.

```
def counter():
    n = 0
    def inc():
        nonlocal n
        n += 1
        return n
    return inc
```

15. What is a closure?

A closure is a nested function that remembers variables from its enclosing scope even after the outer function has returned. Closures power decorators, callbacks, and function factories.

```
def multiplier(k):
    return lambda x: x * k

double = multiplier(2)
double(5)   # 10
```

16. What's the difference between an iterable and an iterator?

An iterable has an __iter__ method that returns an iterator (lists, strings, dicts). An iterator has __next__ and remembers its position, raising StopIteration when exhausted. A for loop calls iter() on an iterable and then next() repeatedly.

```
it = iter([10, 20])
next(it)   # 10
next(it)   # 20
next(it)   # StopIteration
```

17. What is a lambda function?

A lambda is a small anonymous function limited to a single expression. It is handy for short callbacks such as sort keys, but a regular def is better for anything non-trivial because it is named and can hold statements and docstrings.

```
sorted(users, key=lambda u: u['age'])
```

18. What do map(), filter(), and reduce() do?

map applies a function to every item; filter keeps items for which a function returns True; functools.reduce folds a sequence into a single value. In modern Python, comprehensions are usually more readable than map and filter.

```
from functools import reduce
list(map(str, [1, 2]))            # ['1', '2']
list(filter(None, [0, 1, 2]))     # [1, 2]
reduce(lambda a, b: a + b, [1,2,3])   # 6
```

19. What is the difference between __init__ and __new__?

__new__ creates and returns the new instance (it is a static method that receives the class). __init__ then initialises that already-created instance. You override __new__ rarely, for singletons or for subclassing immutable types.

20. Explain instance methods, class methods, and static methods.

An instance method takes self and works with instance state. A @classmethod takes cls and works with class state, often as an alternative constructor. A @staticmethod takes neither and is just a function namespaced inside the class.

```
class User:
    def __init__(self, name): self.name = name
    @classmethod
    def from_email(cls, e): return cls(e.split('@')[0])
    @staticmethod
    def is_valid(e): return '@' in e
```

21. What does @property do?

It lets you expose a method as an attribute, so you can add validation or computed values behind normal attribute syntax. You can add a setter with @name.setter. It lets you start with a plain attribute and later add logic without breaking callers.

```
class Circle:
    def __init__(self, r): self._r = r
    @property
    def area(self): return 3.14159 * self._r ** 2
```

22. What is the difference between __str__ and __repr__?

__str__ returns a readable string for end users (used by print and str). __repr__ returns an unambiguous developer-oriented string, ideally one that could recreate the object (used in the REPL and logs). If only __repr__ is defined, it is also used as the fallback for str.

23. What is the MRO, and how does inheritance resolve methods?

The Method Resolution Order is the order in which Python searches base classes for an attribute. It uses C3 linearization, which keeps each class before its parents and preserves the order of bases. Inspect it with Class.__mro__ or Class.mro(). It also solves the diamond problem predictably.

```
class A: pass
class B(A): pass
class C(A): pass
class D(B, C): pass
D.__mro__   # D, B, C, A, object
```

24. What does super() do?

super() returns a proxy that delegates method calls to the next class in the MRO, not simply 'the parent'. It is used to call parent implementations (especially __init__) and is essential for cooperative multiple inheritance.

25. What are abstract base classes?

An ABC, defined with the abc module, declares methods that subclasses must implement. Instantiating a class with unimplemented @abstractmethod methods raises TypeError. They enforce interfaces in a codebase.

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self): ...
```

26. What is duck typing?

Python cares about what an object can do, not what type it is: 'if it walks like a duck and quacks like a duck, it's a duck'. Any object with the needed methods works, which is why functions accept any file-like or iterable object without type checks.

27. What are context managers and the with statement?

A context manager guarantees setup and cleanup around a block. It implements __enter__ and __exit__, or can be written with the contextlib.contextmanager decorator. Typical uses are files, locks, and database transactions.

```
from contextlib import contextmanager

@contextmanager
def opened(path):
    f = open(path)
    try:
        yield f
    finally:
        f.close()
```

28. How does try / except / else / finally work?

try holds the risky code. except handles specific errors. else runs only if no exception occurred. finally always runs, whether or not an exception happened, and is used for cleanup. Catch specific exceptions rather than a bare except.

```
try:
    x = int(s)
except ValueError:
    x = 0
else:
    print('parsed ok')
finally:
    print('done')
```

29. How do you create a custom exception?

Subclass Exception (or a more specific built-in). Custom exceptions make error handling precise and let callers catch your library's errors specifically. Use raise ... from e to preserve the original cause.

```
class PaymentError(Exception):
    pass

raise PaymentError('card declined')
```

30. What is EAFP vs LBYL?

EAFP means 'Easier to Ask Forgiveness than Permission': try the operation and handle exceptions. LBYL means 'Look Before You Leap': check conditions first. Python style favours EAFP, which also avoids race conditions between the check and the use.

Memory, Performance and Concurrency

31. How does Python manage memory?

CPython uses private heap memory managed by the interpreter. Objects are freed primarily through reference counting, and a cyclic garbage collector detects and cleans up reference cycles that counting alone cannot free. Small objects are served by an internal allocator (pymalloc) for speed.

32. What is __slots__ and when would you use it?

Declaring __slots__ tells Python to store attributes in a fixed layout instead of a per-instance __dict__. This saves significant memory when you create millions of small objects and prevents adding arbitrary new attributes.

```
class Point:
    __slots__ = ('x', 'y')
    def __init__(self, x, y):
        self.x, self.y = x, y
```

33. Is Python compiled or interpreted?

Both. CPython first compiles source to bytecode (stored in __pycache__ as .pyc files), then the Python virtual machine interprets that bytecode. It is not compiled to native machine code by default.

34. Threading vs multiprocessing vs asyncio: when do you use each?

Threading suits I/O-bound work with blocking libraries. Multiprocessing suits CPU-bound work because each process has its own interpreter and GIL. asyncio suits large numbers of concurrent I/O tasks (network calls, sockets) using a single thread and cooperative scheduling.

35. How do async and await work?

async def defines a coroutine. await pauses it until an awaitable finishes, handing control back to the event loop so other tasks can run. Concurrency comes from the event loop switching tasks at await points; nothing runs in parallel on one thread.

```
import asyncio

async def fetch(n):
    await asyncio.sleep(1)
    return n

async def main():
    print(await asyncio.gather(fetch(1), fetch(2)))

asyncio.run(main())   # about 1 second total
```

36. How can you work around the GIL?

Use multiprocessing or concurrent.futures.ProcessPoolExecutor for CPU-bound work, move hot loops into C extensions or libraries that release the GIL (NumPy, Cython), or use alternative runtimes. Recent CPython versions also offer an optional free-threaded build.

37. What are dataclasses?

The @dataclass decorator auto-generates __init__, __repr__, and __eq__ from annotated fields, removing boilerplate for classes that mainly hold data. Options include frozen=True for immutability and field(default_factory=list) for mutable defaults.

```
from dataclasses import dataclass, field

@dataclass
class Order:
    id: int
    items: list = field(default_factory=list)
```

38. What are type hints, and does Python enforce them?

Type hints annotate expected types (def f(x: int) -> str). Python does not enforce them at runtime; they are used by static checkers such as mypy and pyright, by IDEs, and by libraries like Pydantic and FastAPI.

39. f-strings vs format() vs % formatting?

f-strings (3.6+) are the most readable and fastest: they evaluate expressions inline. str.format() uses placeholders and is useful for reusable templates. % formatting is the legacy style. Prefer f-strings.

```
name, n = 'Nilesh', 3.14159
f'{name} {n:.2f}'    # 'Nilesh 3.14'
```

40. How does Python sort, and what's the difference between sorted() and list.sort()?

Python uses Timsort, a stable hybrid of merge sort and insertion sort with O(n log n) worst case and O(n) on nearly sorted data. sorted() returns a new list from any iterable; list.sort() sorts in place and returns None. Both accept key= and reverse=.

41. What are enumerate() and zip() used for?

enumerate gives (index, item) pairs while looping. zip pairs items from several iterables and stops at the shortest (use itertools.zip_longest or strict=True in 3.10+ to change that).

```
for i, name in enumerate(names, start=1): ...
for n, s in zip(names, scores): ...
```

42. What useful classes are in the collections module?

Counter counts hashable items; defaultdict supplies a default value for missing keys; deque is a fast double-ended queue with O(1) appends and pops at both ends; namedtuple creates lightweight immutable records; OrderedDict and ChainMap cover niche needs.

```
from collections import Counter, defaultdict
Counter('banana').most_common(1)   # [('a', 3)]
groups = defaultdict(list)
groups['x'].append(1)
```

43. What is itertools and why use it?

itertools provides memory-efficient, lazy iterator building blocks such as chain, islice, groupby, product, permutations, combinations, and cycle. They let you express complex iteration without building intermediate lists.

44. What do functools.lru_cache and partial do?

lru_cache memoizes function results so repeated calls with the same arguments return instantly (arguments must be hashable). partial fixes some arguments of a function to create a new, simpler callable.

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fib(n):
    return n if n < 2 else fib(n-1) + fib(n-2)
```

45. What is pickling?

Pickling serialises a Python object to bytes (pickle.dumps) and unpickling restores it (pickle.loads). It is Python-specific and unsafe with untrusted data because loading can execute arbitrary code. Use JSON for data exchanged with other systems.

46. What is the difference between a module and a package, and what does if __name__ == '__main__' do?

A module is a single .py file; a package is a directory of modules (traditionally with __init__.py). The __name__ guard runs code only when the file is executed directly, not when it is imported.

```
if __name__ == '__main__':
    main()
```

47. What are virtual environments and why use them?

A virtual environment is an isolated Python install with its own packages, so different projects can use different dependency versions without conflict. Create one with python -m venv .venv, and track dependencies in requirements.txt or pyproject.toml.

48. What are unpacking and the walrus operator?

Unpacking assigns items from an iterable to several names at once, including starred catch-alls. The walrus operator := (3.8+) assigns a value inside an expression, which avoids computing something twice.

```
first, *rest = [1, 2, 3, 4]
if (n := len(rest)) > 2:
    print(n)
```

49. Why is repeated string concatenation slow, and what is the fix?

Strings are immutable, so s += x in a loop can create a new string each time, which can be O(n squared) overall. Collect pieces in a list and call ''.join(parts) once, which is linear.

```
parts = []
for w in words:
    parts.append(w)
text = ' '.join(parts)
```

50. What is integer caching and string interning?

CPython pre-creates and reuses small integers (-5 to 256) and interns some strings, so different names can point to the same object. This is an implementation detail; never rely on it, always use == for value comparison.

```
a, b = 256, 256
a is b    # True
a, b = 1000, 1000
a is b    # implementation-dependent
```

51. What is monkey patching?

Changing or extending a class or module at runtime, for example replacing a method. It is useful in tests and quick hotfixes but makes code hard to reason about, so use it sparingly.

52. What are metaclasses?

A metaclass is the class of a class: it controls how classes themselves are created (type is the default). They are used by ORMs and frameworks to customise class creation, but most problems are better solved with decorators or __init_subclass__.

53. How do you find and fix slow Python code?

Measure first. Use cProfile or line_profiler to find hotspots and timeit for micro-benchmarks. Then pick better algorithms and data structures, cache results, use built-ins and comprehensions, vectorise with NumPy, or move hot paths to C extensions or multiprocessing.

```
python -m cProfile -s cumtime script.py
```

54. What is the difference between append() and extend()?

append adds its argument as a single element at the end of the list, even if that argument is itself a list. extend iterates over its argument and adds each item individually. Both modify the list in place and return None.

```
a = [1, 2]
a.append([3, 4])   # [1, 2, [3, 4]]
b = [1, 2]
b.extend([3, 4])   # [1, 2, 3, 4]
```

55. How do you test Python code? unittest vs pytest, and what is mocking?

unittest is the built-in xUnit-style framework. pytest is the popular third-party runner with plain assert statements, fixtures, and parametrization. Mocking (unittest.mock) replaces real dependencies such as APIs or databases with controllable fakes so tests are fast and isolated.

```
from unittest.mock import patch

def test_fetch():
    with patch('app.requests.get') as m:
        m.return_value.json.return_value = {'ok': True}
        assert fetch() == {'ok': True}
```